CHAPTER 3 | OPERATING SYSTEMS

3.1 What an Operating System Is — Types and Kernel

Operating System: the program that acts as the **bridge between the computer user and the computer hardware**, and the system software that **determines and manages the execution order of programs**. Its functions cover **process management, memory management, file management and security — all of them**. Its services include resource management, inter-process communication and program execution — but **translating source code into machine code is the compiler's job, never an OS service**. A good OS shows **high throughput and low turnaround time**.

OS type	Identity
Batch → Time-sharing	Batch ran jobs with no interaction; **time-sharing marked the beginning of interactive computer communication** — each user gets CPU slices in rotation. **When a time slice expires the process goes Running → Ready** (never Blocked)
Real-time	**Well-defined fixed time constraints**; two types — **hard and soft**; **always runs on embedded systems** (controllers, medical devices)
Distributed	Benefits: scalability, resource sharing, fault tolerance — **high communication latency is the cost, not a benefit**; consistency across nodes is kept by **replication and synchronisation**. Note the pairing: **parallel processing = tightly coupled; distributed = loosely coupled** (the exam statements reversed both — both false)
Multiprogramming / multi-user	**Multiprogramming = more than one process resides in memory** (raises CPU utilisation). **Multi-user = several users on the same machine at once**; multitasking = several programs; **multiprocessing = two or more CPUs shared by programs; multithreading = a program broken into smaller parts loaded as needed**
Others	Network, embedded, mobile — all valid types. **LDAP is a protocol, not an OS** (MS-DOS, iOS, Linux are). There was never a **Windows 96**

Kernel: the **innermost layer of the OS, always resident in main memory** — the memory-resident components ARE the kernel. It is the **first part of the OS loaded during booting** and remains for the whole session; the false claim is that it is a collection of separately loadable programs. **Privileged instructions execute only in kernel mode**; a **user↔kernel mode switch costs LESS than a process context switch** (the address space does not change).

Kernel design	Asked facts
Monolithic	**User and kernel services share ONE address space** ("separate address space" is the false statement); hard to extend; **Linux and Windows are grouped as monolithic** in the tested pairing
Microkernel	Smaller size; services separated; needs more code to design
Hybrid	**OS X (XNU = Mach microkernel + BSD)**; Windows NT also hybrid in modern classification

A **virtual machine abstracts the hardware of one computer into several execution environments**, letting multiple operating systems run concurrently (the hypervisor manages them). A **formal protection model provides a mathematical framework for analysing resource allocation** (access matrix). **32-bit vs 64-bit: a 32-bit OS can address only 2^{32} bytes = 4 GB of RAM**; a 64-bit OS far more. **Spooling** = output dumped to a reserved disk area instead of printing directly (print queues); **an interrupt is what a peripheral generates when it needs immediate OS attention**.

Exam Points — 3.1

- ✦ OS = user–hardware bridge; manages execution order; process + memory + security management; compilation is NOT an OS service; good OS = high throughput + low turnaround.
- ✦ Time sharing began interactive computing; quantum expiry → Ready. RTOS: hard + soft, fixed deadlines, embedded. Distributed: latency is not a benefit; replication + synchronisation keep consistency. Multiprogramming = many processes in memory.
- ✦ Kernel: innermost, memory-resident, loads first. Privileged instructions = kernel mode only; mode switch cheaper than context switch. Monolithic = one address space (Linux); OS X = hybrid; microkernel = small + separated.
- ✦ LDAP is not an OS; no Windows 96; Defender = anti-malware; Programs and Features uninstalls; Windows interface = GUI. 32-bit ceiling = 4 GB. VM = one machine, many environments. Spooling = disk-buffered output.

3.2 Processes, PCB and System Calls

Process: a program in execution — the program itself is passive; the process is active. The kernel tracks each process through its **Process Control Block (PCB)**: process ID, state, program counter, registers, memory limits, open files. **A context switch is essentially saving one PCB and loading another** — the kernel saves the old process's context in its PCB, and the switch time is **pure overhead** (both statements true). **The TLB need NOT be saved** on a context switch (it is flushed); registers and the program counter must be.

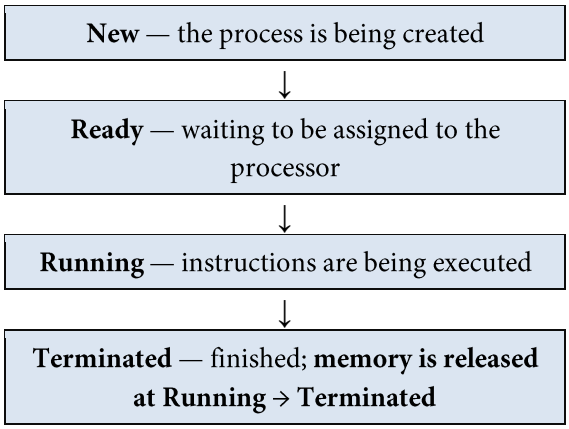


Fig. 3.1 — The main lifecycle. A Running process may also move to Waiting/Blocked (for I/O) and return to Ready when the event completes.

(1) Valid states: new, ready, running, waiting (blocked), terminated — "Excite" is not a state. **Blocked → Running is impossible** (the dispatcher picks only from the ready queue); Blocked → Ready happens when the awaited I/O completes.

(2) **Zombie** = a process that has completed but still shows an entry in the process table (awaiting the parent's reap); an orphan's parent died first.

(3) **fork()** returns **−1 on failure, 0 to the CHILD, and the child's PID to the parent**. **getpid() returns the current process's ID** (getppid() the parent's).

System-call category	Examples asked
Process control	**end, abort, create**, load, execute, **wait for time**
File manipulation	create, open, **read, write, reposition**, close, delete — file manipulation = operating on file content
Device manipulation	request device, release device
Information maintenance	get time, get process attributes
Communication	**attach or detach remote devices**, send/receive messages

"**Data compilation**" is not a system-call category. Windows file calls: CreateFile, ReadFile, WriteFile and **CloseHandle — CloseFile() does not exist**; CloseHandle takes **a handle to the resource**, and skipping it leaks resources (**resources are not released**), it does not corrupt the file. **ReadConsole** reads from the console/terminal (ReadFile for files/devices). Unix **open** modes: O_RDONLY, O_WRONLY, O_RDWR — **O_WRRK is invented**. **write returns ssize_t** (signed, so −1 can report failure). A **trap is software-generated** (raised by the running instruction — divide-by-zero, system call); calling it a hardware-generated interrupt is the false statement; the OS is interrupt-driven, the bootstrap loads at power-up, and the kernel runs at all times.

Exam Points — 3.2

- ✦ Process = program in execution; PCB holds all process information; context switch = save/restore PCBs, pure overhead; TLB not saved.
- ✦ States: new/ready/running/waiting/terminated; Blocked→Running impossible; I/O completion → Ready; memory freed on termination; zombie = done but still in the table.
- ✦ fork: −1 fail, 0 child, PID to parent; getpid = current PID. System-call classes: process control (end/abort/create/wait-for-time), file, device, information, communication (attach/detach remote); no 'data compilation'.
- ✦ Windows closes with CloseHandle (no CloseFile); handle leak if skipped; ReadConsole for terminals. O_WRRK invalid; write returns ssize_t. Trap = software-generated.

3.3 Threads

Thread: the **lightweight process** — a subset of a process, a single flow of execution inside it. **Threads of one process SHARE the code section, data section and open files**; each thread keeps **its own thread ID, program counter, registers and stack** — in fact a separate **user stack AND kernel stack** per thread. **Threads encapsulate concurrency within a process; processes encapsulate protection within the system** (both true). A thread is a **strictly private resource of its process** — not visible to other processes.

(1) Advantages over processes — **all four**: less time for creation, less time for termination, less time for switching, and **no kernel involvement in communication** among threads of one process.

(2) Genuine benefits: improved responsiveness, better resource utilisation, simpler model. **NOT a benefit: faster execution on a single-core CPU** — one core still runs one thread at a time.

(3) **User-level threads are FASTER to create and manage than kernel-level threads** (no kernel calls) — the claim that kernel threads are faster is the false one. The cost: one blocking call can block the whole process; **scheduler activations** (kernel upcalls to the thread library) solve that communication problem.

(4) **Asynchronous cancellation terminates the target thread immediately**; deferred cancellation only sets a flag for the target to check.

Exam Points — 3.3

✦ Thread = lightweight process; shares code + data + files; private: ID, PC, registers, user + kernel stacks; threads = concurrency, processes = protection; a thread is private to its process.

✦ All four thread advantages hold; no speed gain on a single core. User-level threads faster to manage; scheduler activation = library–kernel communication; asynchronous cancellation kills at once.

3.4 CPU Scheduling

The **scheduler selects** the next process; the **dispatcher actually allocates the CPU to it** — performing the context switch (dispatch latency = the time to stop one process and start another). The false claim is that "the dispatcher is not used in CPU scheduling." The three schedulers: **long-term = job scheduling** (admission, degree of multiprogramming), **medium-term = swapping** (also called load control — keeping CPU time evenly shared), **short-term = CPU scheduling** (runs many times a second).

Metric	Meaning
Throughput	**Number of processes completed per unit time** — the work a system does per unit time (asked in three papers)
Turnaround time	**The SUM of periods spent waiting to get into memory + waiting in the ready queue + executing + doing I/O** — the whole life of the job
Waiting / Response time	Waiting = time in the ready queue only; response = submission to FIRST response

Two computed measures:

$$CPU\ efficiency = useful\ time \div (useful + switch\ overhead) \qquad \rho = \lambda \div \mu$$

Worked: run 6 units + switch 4 units → efficiency 6/10 = **0.6**. Arrivals 10/min, service 3 s (μ = 20/min) → load ρ = **0.5**.

Algorithm	Asked facts
FCFS	Serves in **arrival order**; non-preemptive — average waiting time depends mainly on **arrival order**; a lone arriving process starts at its **arrival time**
SJF / SJN	**Minimum average waiting time (optimal)** and **maximum throughput** — but needs run times in advance; SRTF (preemptive form) **may cause starvation**
Priority	Multilevel queue scheduling is its close relative (one queue per priority class); risk = **indefinite blocking (starvation) of low-priority jobs, cured by AGING** — aging gradually raises the priority of waiting low-priority jobs (only that statement is true)
Round Robin	**Better response time than FCFS**; as the quantum grows turnaround **varies irregularly** (RR → FCFS at large quantum); a lone n-quanta job makes **n – 1 context switches** (10 units/quantum 1

	→ **9**; 14 units/quantum 2 → **6**); P1=24, P2=3, P3=3, q=4 → **P1 P2 P3 P1 P1 P1 P1 P1**; jobs a… with 1 ms slices → job a completes at **10**

(1) **Starvation is a problem, not an algorithm. Busy waiting is the undesirable wait** (burns CPU while achieving nothing).

(2) Sorting the families: CPU — FCFS, SJF, RR, priority; disk — SSTF, SCAN, C-SCAN, LOOK; pages — FIFO, LRU, optimal. So **C-SCAN is not CPU scheduling; SJF is the CPU algorithm** in a mixed list.

(3) Non-preemptive priority interrupts 4,7,1,3,0,2,6 (two arriving during each service, low number = high priority) are handled **4, 1, 0, 2, 3, 6, 7** — finish the current one, then pick the highest priority waiting.

Exam Points — 3.4

✦ Scheduler decides, dispatcher switches (dispatch latency). Long = job, medium = swapping/load control, short = CPU.

✦ Throughput = jobs per unit time; turnaround = memory wait + ready wait + execution + I/O. Efficiency worked 0.6; load worked 0.5.

✦ FCFS → arrival order. SJF optimal (min waiting, max throughput); SRTF starves. Priority → aging cures starvation. RR: better response than FCFS; n−1 switches; both RR sequences worked.

✦ Starvation is not an algorithm; busy waiting is the undesirable wait; C-SCAN/LOOK/LRU are not CPU scheduling; interrupt order 4,1,0,2,3,6,7.

3.5 Process Synchronisation

Critical section problem: when processes **share resources**, their access must be serialised — mutual exclusion problems occur only **among processes that share resources**. A correct solution must satisfy three conditions together: **mutual exclusion, progress and bounded waiting**. The critical-region mechanism exists for **process synchronisation**.

Semaphore: an integer variable accessed, apart from initialisation, **only through two standard atomic operations: wait() and signal()** (Dijkstra's P and V). Semaphores solve **mutual exclusion** (a binary semaphore = mutex) and can implement **bounded waiting**.

$$wait(S)\text{: } while\ S \leq 0\ do\ nothing;\ S = S - 1 \qquad signal(S)\text{: } S = S + 1$$

(1) **Peterson's solution works for exactly 2 processes** (Lamport's bakery algorithm handles any number).

(2) The three classic synchronisation problems: **bounded buffer, readers-writers, dining philosophers — Belady's anomaly does not belong** (it is paging).

(3) A **memory transaction is a sequence of memory read-write operations that is atomic. Semaphores and monitors are system objects that are neither files nor directories.**

Exam Points — 3.5

✦ Critical-section solution = mutual exclusion + progress + bounded waiting. Mutual exclusion arises among resource-sharing processes.

✦ Semaphore: atomic wait()/signal() only; wait decrements after the S ≤ 0 loop, signal increments; solves mutual exclusion; supports bounded waiting.

✦ Peterson = 2 processes. Classic problems: bounded buffer, readers-writers, dining philosophers. Memory transaction = atomic read-write sequence; semaphore/monitor are neither files nor directories.

3.6 Deadlock

A deadlock needs **all four Coffman conditions at once**: **mutual exclusion** (at least one resource non-sharable — that is the condition the exam names), **hold-and-wait**, **no preemption**, and **circular wait**. "Preemption" itself is NOT a condition — allowing preemption is a cure. The four strategies, in order of when they act:

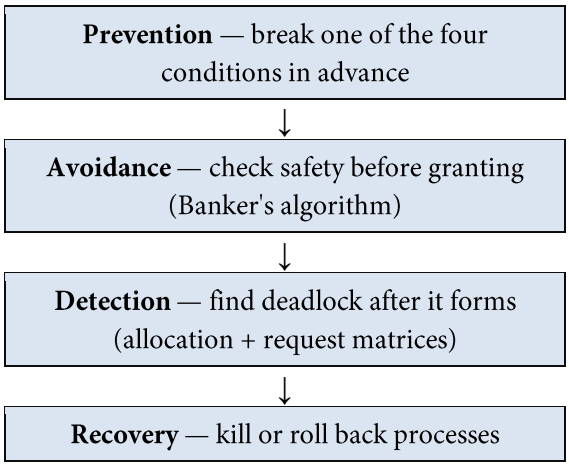


Fig. 3.2 — The deadlock-handling ladder.

(1) **Valid prevention schemes**: release all resources before requesting new ones, and request everything before execution (both break hold-and-wait); **number the resources and request only in increasing order** (breaks circular wait). "Never request a resource after releasing any resource" breaks nothing — the invalid scheme.

(2) **Banker's algorithm = deadlock AVOIDANCE**, used when resource types have **multiple instances** (graphs handle single-instance). The **resource matrix represents resource allocation**; the **resource-allocation graph represents the state of system resources and processes**.

(3) **Cycle rules**: a cycle in the **wait-for graph = deadlock** (true); a cycle in the resource-allocation graph is only **necessary, not sufficient** when types have several instances (so "RAG cycle indicates deadlock" is false); the Coffman list is true — answer B and C.

The guarantee formula — with P processes each needing N units, deadlock is impossible when:

$$total\ units \ \geq \ P \times (N - 1) + 1$$

Worked: 3 processes × 2 units → 3×1+1 = **4 units** minimum. 7 tape drives, need 2 each: n × 1 < 7 … works up to **n = 3** processes. Banker's worked: 10 units; X(max 8, has 4), Y(7, 1), Z(5, 3); 2 free → **only Z's request for 2 may be granted** (Z finishes, releases 5, then X, then Y).

Exam Points — 3.6

✦ Coffman: mutual exclusion (non-sharable), hold-and-wait, no preemption, circular wait; 'preemption' is not a condition.

✦ Resource ordering breaks circular wait; release-all and request-all break hold-and-wait; the never-request-after-release scheme is invalid.

✦ Banker's = avoidance, multi-instance; wait-for cycle = deadlock, RAG cycle only necessary. Formula P(N−1)+1: worked 4 units and n = 3; Banker's worked → grant only Z. Aging prevents starvation.

3.7 Memory Management — Allocation, Fragmentation, Paging

Memory management **keeps track of which parts of memory are in use AND allocates memory to the programs to be loaded** (both). **Contiguous allocation** places each process in one unbroken region next to the others — fast access, so it suits **real-time systems, CD-ROMs and video streaming**, but not **general-purpose systems with dynamic file sizes** (a contiguous file cannot grow in place).

Placement strategies: first fit (first adequate hole), **best fit** (smallest adequate hole — its drawback: it **creates many small unusable holes**; its strength: it preserves big holes — the 6/15/12 KB free list serving requests 10/2/14 KB succeeds **only under best fit**), **worst fit** (largest hole), next fit — **"not fit" does not exist.**

Internal fragmentation	External fragmentation
Waste **INSIDE** an allocated partition — 60k in an 80k partition wastes 20k internally; the standing problem of **fixed partitions**	Free memory scattered in holes **BETWEEN** allocations — holes a and b with a + b > c but each < c cannot serve c
Cure: smaller/variable partitions, paging	**Cure: COMPACTION** — sliding processes together so all free memory forms one block; worked: 500 KB with 100 + 130 used → 270 KB free after compaction → smallest deniable request = **271 KB**

(1) Coalescing (buddy system): adjacent free buddies are **quickly combined into larger segments** — the buddy system's advantage.

(2) Base register = where the partition starts; limit register = its size. Partitions of $2^{16}/2^{24}/2^{32}$ → limit needs **32 bits** (largest); 2^{24} memory in fixed 65,536-byte partitions → limit needs only **16 bits.**

(3) Swapping = copying a process from memory to **disk** (swap space lives on disk) to free memory — removal of inactive programs; swap out = memory→disk, swap in = the reverse; done by the medium-term scheduler.

Paging and Segmentation

Paging divides logical memory into **pages** (fixed-size blocks of VIRTUAL memory) and physical memory into **frames** — same size. The **page table maps virtual (logical) addresses to physical addresses** — that translation is "page table mapping." Address arithmetic:

$$offset\ bits = log_2(page\ size) \qquad logical\ address\ bits = log_2(blocks) + log_2(block\ size)$$

Worked: 4 KB page → **12 offset bits** (physical space size is irrelevant to the offset). 4 pages × 2048 words → 2 + 11 = **13 bits**. 64 segments × 2 KB → 6 + 11 = **17 bits.**

Segmentation: each segment-table entry holds a **base = the segment's starting address in PHYSICAL memory** and a limit = its length. The **Volume Control Block** (file-system implementation) holds partition-level metadata — **the number of blocks in the partition.**

Exam Points — 3.7

✦ Contiguous allocation: one block per process; suits real-time/CD-ROM/streaming, not dynamic file sizes. Best fit → many tiny holes; the 6/15/12 worked case fits only best fit; 'not fit' invalid.

✦ Internal = waste inside (fixed partitions); external = scattered holes; compaction cures external (271 KB worked). Coalescing merges buddies. Limit register: 32-bit and 16-bit worked answers.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

✦ Page = fixed block of virtual memory; frame = physical; page table maps virtual→physical. Offset 12 bits for 4 KB; logical address 13 and 17 bits worked. Segment base = physical start. Swap space = disk; kernel = memory-resident core.

3.8 Virtual Memory and Page Replacement

Virtual memory: extends physical memory using secondary storage — a **temporary** storage area for data larger than RAM, combining **hard disk (to hold pages) with RAM (to speed access)**. Its size depends on the **address bus**, not on installed RAM. **Demand paging** brings a page in **only when it is needed** — which is what lets a program be **larger than physical memory**.

(1) **Page fault = referencing a page not currently in memory** (not an error). **Thrashing = the rate at which page faults occur becomes excessive** — the CPU spends its time paging.

(2) Effective access time weighs hits and misses:

$$EAT = h \times t_{hit} + (1 - h) \times t_{miss}$$

Worked: TLB hit 100 ns (80%), miss 400 ns → 0.8×100 + 0.2×400 = **160 ns**.

Algorithm	Rule and asked results
FIFO	Evicts the **oldest** page — **suffers Belady's anomaly** (more frames can mean MORE faults; the anomaly belongs to page replacement, and FIFO alone among the asked options). Worked: 4 frames, 1,3,4,4,3,2,1,7,5,6 → **7 faults**; 3 frames, 3,4,2,3,9,1,6,3,4,9 → **9**; 3 frames, 7,0,1,2,0,3,0,4,2,3 → **9**
LRU	Evicts the **least recently used** — a stack algorithm, **immune to Belady**. Worked: 4 frames, 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1 → **7 faults**; 3 frames, 7,0,2,0,0,7,1 → **4**
Optimal (OPT)	Evicts the page **not used for the longest future time** — fewest possible faults, unimplementable, the benchmark. Worked: 3 frames, 4,7,6,1,7,6,1,2,7,2 → **5 faults**
LFU / **MFU**	MFU evicts the largest count, reasoning **a page with a small count was probably just brought in and is still to be used**

Exam Points — 3.8

✦ VM = disk-extended RAM, temporary, disk + RAM combination; size set by the address bus; demand paging = load on reference; programs may exceed RAM.

✦ Page fault = page absent; thrashing = excessive fault rate. EAT worked = 160 ns.

✦ FIFO shows Belady's anomaly (LRU/OPT immune — stack algorithms). All six page-fault workings above are direct paper questions; OPT = farthest-future eviction.

3.9 File Systems and Directories

(1) **File permissions** decide who can read, write or execute a file. **FAT = File Allocation Table** (asked in three papers) — it stores **entries about files and their characteristics** (the cluster chains). **NTFS supports compression** — the statement that it does not is the false one; NTFS uses small clusters efficiently and improved on its precursor. **NTFS and FAT32 relate to FORMATTING** (laying a file system on a volume).

(2) **NTFS, FAT32 and HFS+ are all real file systems**; the **Linux default is Ext4** (Ext2 → Ext3 journaling → Ext4). **Coda's architecture is based on the Andrew File System.**

(3) **Links** make a file or directory visible in multiple parts of the file tree. In an **acyclic-graph directory, sharing is implemented with symbolic (soft) links** — a soft link stores the target's path (can cross file systems); a hard link shares the inode (barred for directories to avoid cycles).

(4) **Defragmentation** organises each file into contiguous regions and merges free space; **compression** makes files take less space — the tested pairing. Access methods — **sequential, direct, random, associative — characterise the memory system.**

Exam Points — 3.9

- ✦ Permissions control r/w/x. FAT = File Allocation Table, records files and their blocks. NTFS DOES compress; NTFS/FAT32 → formatting. Linux default = Ext4; Coda ← Andrew FS.
- ✦ Links = multi-location visibility; acyclic-graph sharing = soft links. Defragmentation + compression pairing. Sequential/direct/random/associative = memory access methods.

3.10 Linux and UNIX — Shell, Commands, Directories

Shell: the command interpreter — **a programming language in its own right**; shell scripts **accept arguments and are INTERPRETED, never compiled** ("compiled" is the false statement). Real shells: **Bourne (sh), Bash, C shell, Korn shell** — "Alpha cell" is not one. Default Bash prompt: **$ for a normal user** (# for root).

Command / symbol	Asked meaning
rm / rmdir	**rm deletes files** (rm doc erases files named doc; with * it erases matches); rm -r removes non-empty directories, rmdir empty ones
touch / cat	The two file-CREATING commands: touch makes an empty file; **cat > x creates x and waits for keyboard input** (Ctrl-D ends)
grep family	**grep, egrep, fgrep search content inside files** — grep prints the lines matching a pattern
head / tail	Both default to **10 lines**; **tail -c +512 copies every byte after skipping 511** (+N = from the N-th onward; bare N = last N)
chmod	Takes **permission changes and file names**; default file permission **644 (rw- r-- r--)**, directories 755
Redirection & wildcards	**<, >, >> redirect (! does not)**; ? = one character, * = any characters, [] **= one character from a class**
Shell variables	**$# = number of arguments**, $? = exit status, $$ = PID, $* = all arguments
ls -l types	- file, d directory, l link, **c = character special (device) file**, b block device

Directory map: **/etc = local system configuration files** (asked in three papers); /bin = executable UNIX commands; /dev = device files; **/usr = user programs and libraries — NOT temporary files** (those live in /tmp; the /usr claim was the false statement); /boot = boot files. (The stray Unicode question keyed here belongs with Chapter 1's codes: Unicode gives a unique number to every character on every platform.)

Exam Points — 3.10

- ✦ Shell = programming language; scripts interpreted, take arguments. Shells: Bourne/Bash/C/Korn. Prompt $ (root #).
- ✦ rm deletes; touch + cat create; grep/egrep/fgrep search; head/tail default 10; tail -c +512 skips 511 bytes; chmod = permissions + filenames; default 644.
- ✦ Redirection <, >, >> (not !); [] = character class; $# counts arguments; c = character special file. /etc = configuration; /usr = programs, not temp files.

3.11 Disk Structure, Disk Scheduling and I/O

A hard disk is a **magnetic disk of several platters**; a CD-ROM records **optically** (pits), not magnetically. Timing:

$$access\ time\ =\ seek\ time\ +\ rotational\ latency\ +\ transfer\ time$$

(1) **Seek time** = moving the read/write head in or out to the correct **track/cylinder** (two papers); **rotational delay (latency)** = waiting for the desired **sector** to rotate under the head. The KVS wording — 'position the head on the appropriate sector' — is still keyed **seek time**.

Disk algorithm	Rule
FCFS	Serves in arrival order — no optimisation. Worked: head 60, requests 60,45,21,67 → 15+24+46 = **85**; head 53, requests 98,183,37,122,14,124,65,67 → **640** cylinders
SSTF	Nearest request first — advantage: **average response time decreases**; risk: starvation of far requests
SCAN	The elevator: **one direction to the DISK END, serving all requests, then reverses**
LOOK / C-SCAN	LOOK reverses at the **last request** (not the end); C-SCAN returns to the start and sweeps one way only

A **buffer** is the storage area **compensating for the speed difference between units**; the kernel's **buffer cache stores copies of frequently used file blocks in memory** so repeat reads skip the disk.

Exam Points — 3.11

- ✦ Seek = track positioning; rotational delay = sector wait; access = seek + latency + transfer. Hard disk = platters (magnetic); CD = optical.
- ✦ FCFS worked totals 85 and 640. SSTF lowers average response. SCAN reverses at the disk end; LOOK at the last request.
- ✦ Buffer = speed-difference store; buffer cache = frequently used blocks in memory.

Quick Revision —

1. OS = user–hardware bridge; manages process, memory, files, security; compilation is not its service; good OS = high throughput + low turnaround.
2. Time sharing began interactive computing; slice expiry → Ready. RTOS: hard + soft, embedded, fixed deadlines. Distributed: latency is a cost; replication + synchronisation for consistency.
3. Multiprogramming = many processes in memory; multi-user = many people; multiprocessing = many CPUs; multithreading = program split into parts. Parallel = tightly coupled, distributed = loosely.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

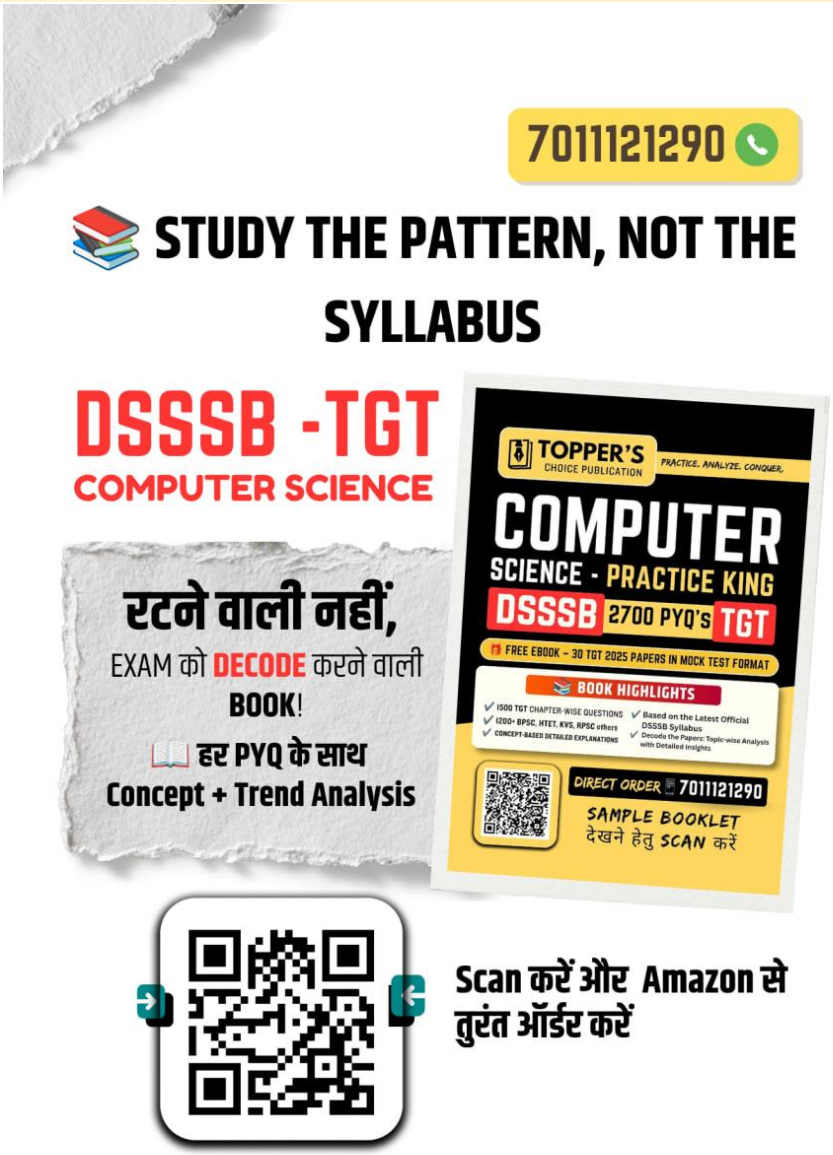
Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

4. Kernel: innermost, memory-resident, loads first, single core program. Privileged instructions = kernel mode. Mode switch < context switch. Monolithic = one address space (Linux); OS X = hybrid; microkernel small + separate.
5. LDAP not an OS; no Windows 96; Defender = anti-malware; Programs and Features = uninstall; 32-bit = 4 GB limit; VM = many environments on one machine; spooling = disk-queued output.
6. Process = program in execution; PCB = its record; context switch = swap PCBs, pure overhead, TLB not saved.
7. States: new, ready, running, waiting, terminated. Blocked→Running impossible; I/O completion → Ready; memory freed on termination; zombie = finished but listed.
8. fork: −1/0/child-PID; getpid = own PID. System calls: process control (end, abort, create, wait-for-time), file, device, information, communication (attach/detach remote). CloseHandle (not CloseFile) takes a handle; leak if skipped.
9. Thread = lightweight process; shares code/data/files; own ID, PC, registers, user + kernel stacks; all four advantages over processes; no single-core speed-up; user threads faster to manage; scheduler activations; asynchronous cancellation = immediate kill.
10. Scheduler picks, dispatcher switches. Long = job admission, medium = swapping/load control, short = CPU.
11. Throughput = completions per unit time; turnaround = memory wait + ready wait + CPU + I/O; response = to first output. Efficiency 6/10 = 0.6; $\rho = \lambda/\mu = 0.5$.
12. FCFS: arrival order. SJF: optimal waiting + max throughput. SRTF: starvation. Priority: starvation → aging. RR: better response; quantum↑ → turnaround irregular; n−1 switches (9; 6); RR sequence P1 P2 P3 P1×5.
13. Starvation is a condition, not an algorithm; busy waiting is the bad wait; SCAN/C-SCAN/LOOK = disk, LRU = pages.
14. Critical section: mutual exclusion + progress + bounded waiting; arises among sharing processes.
15. Semaphore: atomic wait/signal (P/V); wait loops then decrements, signal increments; binary = mutex; Peterson = 2 processes; classics = bounded buffer, readers-writers, dining philosophers.
16. Deadlock = mutual exclusion (non-sharable) + hold-and-wait + no preemption + circular wait, together.
17. Prevention: order resources (kills circular wait); release-all / request-all (kill hold-and-wait). Banker's = avoidance, multi-instance. Wait-for cycle = deadlock; RAG cycle only necessary.
18. Deadlock-free ≥ P(N−1)+1: 3×2 → 4 units; 7 drives, need 2 → n = 3. Banker's worked: grant only Z.
19. Memory management tracks + allocates. Contiguous allocation: real-time/CD/streaming yes, dynamic sizes no.
20. First/best/worst/next fit (no 'not fit'); best fit → tiny holes but saved the 6/15/12 case.
21. Internal fragmentation = waste inside (fixed partitions); external = scattered holes → compaction (271 KB worked). Coalescing merges buddies. Limit register 32-bit / 16-bit worked.
22. Swapping: process ↔ disk swap space; medium-term scheduler; frees RAM.
23. Page = virtual block, frame = physical, same size; page table: virtual → physical. Offset = log2(page size) → 12 bits; logical bits 13 (4×2048) and 17 (64×2KB). Segment base = physical start.
24. VM = disk-extended RAM (temporary; disk + RAM); size from address bus; demand paging = load on demand; programs can exceed RAM.
25. Page fault = page absent; thrashing = excessive fault rate; EAT = 0.8×100 + 0.2×400 = 160 ns.
26. FIFO → Belady's anomaly; LRU/OPT stack algorithms, immune. Fault counts: FIFO 7, 9, 9; LRU 7, 4; OPT 5. MFU logic: small count = just arrived.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

27. Permissions govern r/w/x; FAT = File Allocation Table; NTFS compresses (false statement trap); NTFS/FAT32 → formatting; Linux default Ext4; Coda from Andrew FS.
28. Links show one file in many places; acyclic-graph sharing = soft links (path); hard link = inode. Defragmentation makes contiguous; compression shrinks.
29. Shell: programming language, interpreted scripts, takes arguments; Bourne/Bash/C/Korn; $ prompt.
30. rm deletes; touch/cat create; grep family searches; head/tail = 10 lines; tail -c +512 skips 511 bytes; chmod perms + files; default 644/755.
31. Redirection < > >> (not !); ? one char, * many, [] class; $# = argument count; c = character device in ls -l.
32. /etc config; /bin commands; /dev devices; /usr programs (NOT temp — /tmp); /boot boot.
33. Disk: seek = track, rotational latency = sector, access = seek + latency + transfer; platters magnetic, CD optical.
34. FCFS disk worked 85 and 640; SSTF cuts average response; SCAN to disk end; LOOK to last request.
35. Buffer absorbs speed differences; buffer cache keeps hot blocks in RAM.



DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B